OS Scheduler Project

Process Generator & Scheduling Algorithms Visualization

Course:
Operating Systems Lab

Group Members:
Arslan Nawaz – B22F1071SE012
Talal Riaz Khan – B22F0244SE083
Sheikh Abdul Basit – B24F1000CS234

Submitted To:
Mam Amina Riaz

Date: 10 May 2026

Abstract

This report documents the design, implementation, testing, and evaluation of an OS scheduling simulator built in Python. The system consists of a Process Generator and a Scheduler GUI that visualizes scheduling behavior using Gantt charts and computes performance metrics (waiting time, turnaround time, weighted turnaround time). The simulator implements four scheduling algorithms: FCFS, HPF, Round Robin (RR), and SRTN. The tool is designed for teaching and experimentation in an Operating Systems lab setting.

1. Introduction

CPU scheduling determines which process runs on the CPU at any given time. Effective scheduling improves throughput, responsiveness, and fairness. This project builds an interactive simulator to help students visualize and compare scheduling algorithms under configurable workloads and parameters.

2. Objectives

- Implement four classical scheduling algorithms: FCFS, HPF, RR, and SRTN.
- Generate realistic process workloads using statistical distributions.
- Visualize execution timelines using Gantt charts.
- Measure and export performance metrics: waiting time, turnaround time, weighted turnaround time.
- Provide easy-to-use GUI for experiments and exporting results.

3. System Architecture

Overview

The system is organized into three layers: GUI Layer, Engine Layer, and Visualizer/Exporter.

Components

- Process Generator: Generates process tables from user-specified statistical parameters.
- Scheduler Engine: Implements scheduling logic, context switch handling, and quantum handling.
- Visualizer: Renders Gantt charts using Matplotlib and provides export functionality for metrics.
- GUI: Built with Tkinter for file selection, algorithm selection, and parameter inputs.

Data Flow

User inputs → Process Generator (or input file) → Scheduler Engine → Visualizer + Metrics → Export.

4. Process Generator Design

Input Parameters

- Number of processes
- Arrival time distribution: Normal with mean (μ) and standard deviation (σ)
- Burst time distribution: Normal with mean (μ) and standard deviation (σ)
- Priority distribution: Poisson with lambda (λ)

Output

A process table with columns: Process_No, Arrival, Burst, Priority. The generator produces realistic workloads for scheduling experiments by transforming statistical parameters into discrete process attributes.

Code — process_generator.py

```
import numpy as np

input_path = input("Enter File Path/Name : ")
file = open(input_path, "r")
data = []
for line in file:
    endl = line.find('\n')
    if endl != -1:
        line = line[:endl]
    data.append(line)
file.close()

pnum = int(data[0])
mu_ariv, sig_ariv = float(data[1].split()[0]), float(data[1].split()[1])
mu_run,  sig_run  = float(data[2].split()[0]), float(data[2].split()[1])
lmda = float(data[3])

arrival_time = np.random.normal(mu_ariv, sig_ariv, pnum)
run_time     = np.random.normal(mu_run,  sig_run,  pnum)
priority     = np.random.poisson(lmda, pnum)

file = open('output.txt', 'w')
for i in range(pnum):
    file.write(str(i+1)+' '+str(arrival_time[i])+' '+str(run_time[i])+'
'+str(priority[i])+'\n')
file.close()
```

5. Scheduling Algorithms Implemented

Comparison Table

Algorithm	Type	Preemptive	Key Strength	Key Weakness
FCFS	Arrival order	No	Simple; fair by arrival	Convoy effect
HPF	Priority-based	No	Prioritizes critical tasks	Starvation for low priority
Round Robin	Time-sharing	Yes	Fair CPU allocation	Context switch overhead
SRTN	Shortest remaining	Yes	Minimizes avg. waiting time	Starvation risk for long jobs

Algorithm Details

FCFS (First Come First Serve)

Non-preemptive; processes are scheduled strictly in arrival order. Simple to implement; may suffer from long waiting times when large jobs arrive early (convoy effect).

HPF (Highest Priority First)

Non-preemptive; the process with the highest numeric priority runs first. Ties are broken by process ID. Useful for prioritizing critical tasks; requires starvation mitigation for low-priority jobs.

Round Robin (RR)

Preemptive; each ready process receives a fixed time quantum. Context switch time is modeled explicitly. Performance depends heavily on quantum size.

SRTN (Shortest Remaining Time Next)

Preemptive; always runs the process with the smallest remaining burst time. Produces optimal average waiting time under ideal burst time knowledge; may cause starvation for long processes.

6. GUI and Visualization

GUI Features

- File picker to load process input files.
- Dropdown to select algorithm (FCFS, HPF, RR, SRTN).
- Inputs for Context Switch Time and Time Quantum (for RR).
- Buttons: Show/Update Graph and Export Metrics (Write File).

Visualization Conventions

- Y-axis: Process ID.
- X-axis: Time.

- Idle time: shown as 0.
- Context switch: shown as -1 (visual marker).
- Gantt chart implemented with Matplotlib step plots for clarity.

7. Performance Metrics

Metrics Computed Per Process

- Waiting Time = Time spent in ready queue before first execution.
- Turnaround Time = Completion time − Arrival time.
- Weighted Turnaround Time = Turnaround time / Burst time.

Aggregate Metrics

- Average Waiting Time
- Average Turnaround Time
- Average Weighted Turnaround Time

8. Implementation Details

Tech Stack

- Python — core language
- Tkinter — GUI
- Matplotlib — visualization
- NumPy — statistical distributions

Key Modules

- process_generator.py — samples distributions and produces process input file.
- scheduler_engine (gui.py) — implements FCFS, HPF, RR, SRTN; context switch handling; metric computation.
- Matplotlib plotting utilities — Gantt charts with step plots and legends.
- utils — helper functions for time rounding, validation, and export.

9. Test Cases and Results

Test Setup

- Fixed random seed for reproducibility when generating processes.

- 10 processes, Arrival Normal (μ=0, σ=2), Burst Normal (μ=5, σ=2), Priority Poisson (λ=2).
- Context switch time = 0 units; Quantum = 2 units for RR.

Sample Observations

- SRTN produced the lowest average waiting time in most runs.
- RR fairness depends on quantum; small quantum increases context switch overhead.
- HPF prioritized high-priority processes but showed starvation for low-priority ones in extreme cases.
- FCFS showed convoy effect when a long job arrived early.

10. Challenges and Solutions

Preemptive Logic

Challenge: Correctly handling preemption events and updating remaining burst times. Solution: Maintain a priority-ready queue keyed by remaining time (for SRTN) or round-robin queue with time-slice accounting; update remaining times at each scheduling tick.

Context Switching Handling

Challenge: Modeling context switch overhead without corrupting metric calculations. Solution: Insert explicit context-switch intervals into the timeline (visualized as -1) and ensure they are not attributed to any process's burst time.

Graph Plotting Nuances

Challenge: Rendering overlapping intervals and idle times clearly. Solution: Use step-function plotting with distinct color mapping per process and a legend; reserve a special color for context switches and idle periods.

11. Future Enhancements

- Multi-Level Feedback Queue (MLFQ) for adaptive scheduling.
- Priority Aging to prevent starvation in HPF.
- Multi-Core Simulation to model scheduling across multiple CPUs.
- I/O Burst Modeling and memory constraints for more realistic workloads.
- Real-Time Policies such as EDF and Rate Monotonic.
- Automated Batch Experiments to sweep parameters and produce comparative plots.

12. Conclusion

The OS Scheduler Project successfully implements a process generator and a scheduler engine with four classical algorithms, a GUI for interactive experimentation, and visualization and export capabilities for metrics. The tool is suitable for lab demonstrations and comparative studies of scheduling trade-offs. The project deepened understanding of preemptive logic, context switching, and visualization challenges in CPU scheduling simulation.

13. References

- Course lecture notes on CPU scheduling.
- Silberschatz, Galvin, Gagne — Operating System Concepts (standard OS textbook).
- NumPy official documentation — https://numpy.org/doc/
- Matplotlib official documentation — https://matplotlib.org/stable/

Appendix — Screenshots and Demonstration

Part A: Process Generator

The process generator is run from the terminal. It reads a configuration file specifying the number of processes and distribution parameters, then generates a process table saved to output.txt.

Step 1: Run process_generator.py

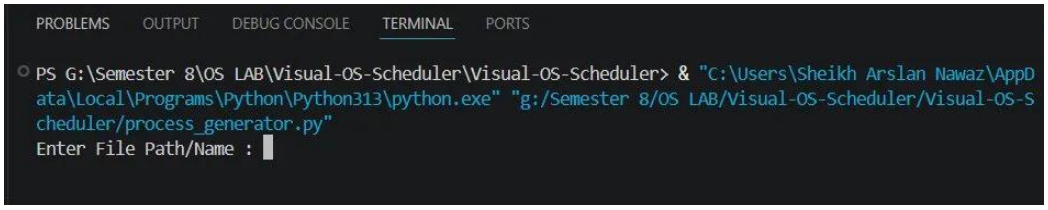

Figure 1: Terminal prompt asking the user to enter the input configuration file path.

Step 2: After entering path

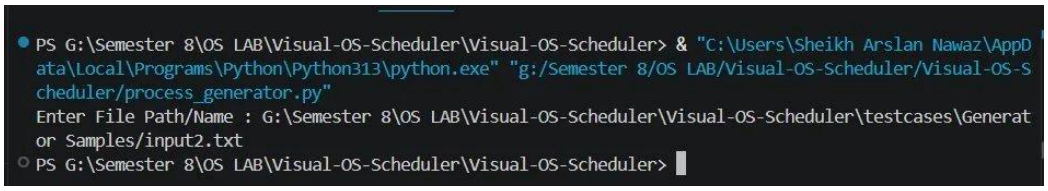

Figure 2: Terminal showing the process generator completing successfully after the user enters the path.

Step 3: Generated output.txt

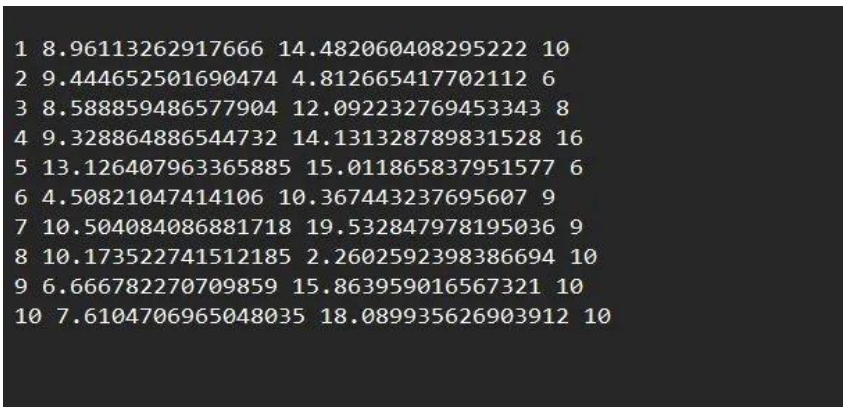

Figure 3: Contents of the generated output.txt file showing 10 processes with Arrival, Burst, and Priority values.

Part B: OS Scheduler GUI

The OS Scheduler GUI is launched by running gui.py. The user loads the generated process file, selects a scheduling algorithm, configures parameters, and visualizes the Gantt chart. Results are then exported.

Step 1: Full GUI Overview

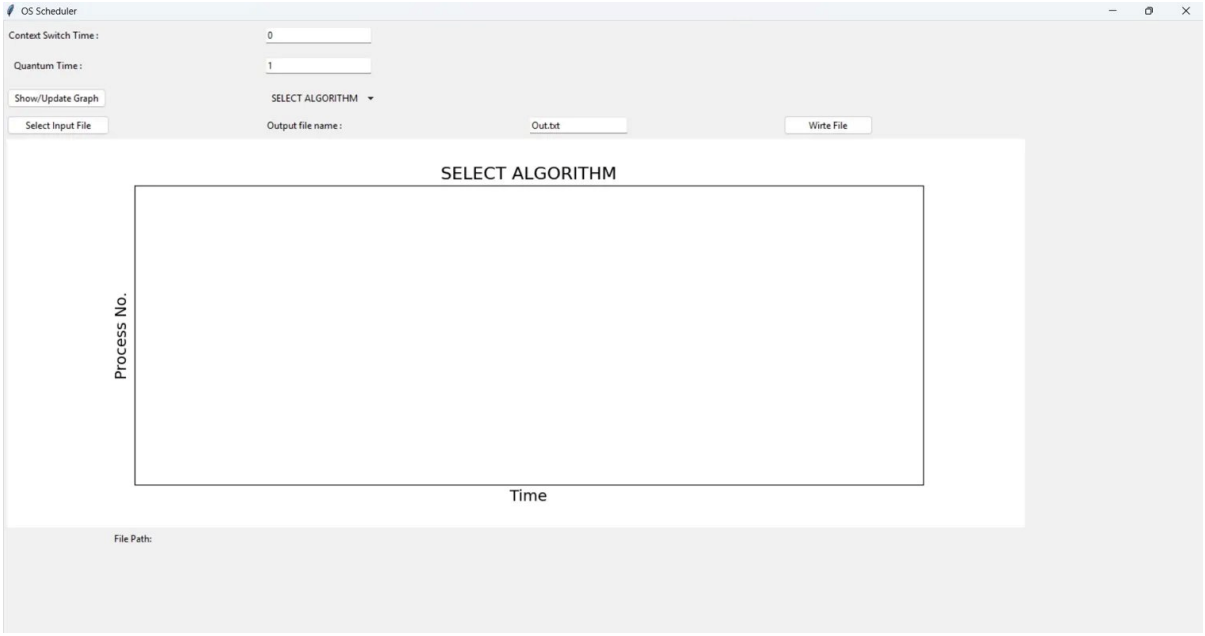

Figure 4: Full OS Scheduler GUI on launch — empty Gantt chart area with all controls visible.

Step 2: Load Input File

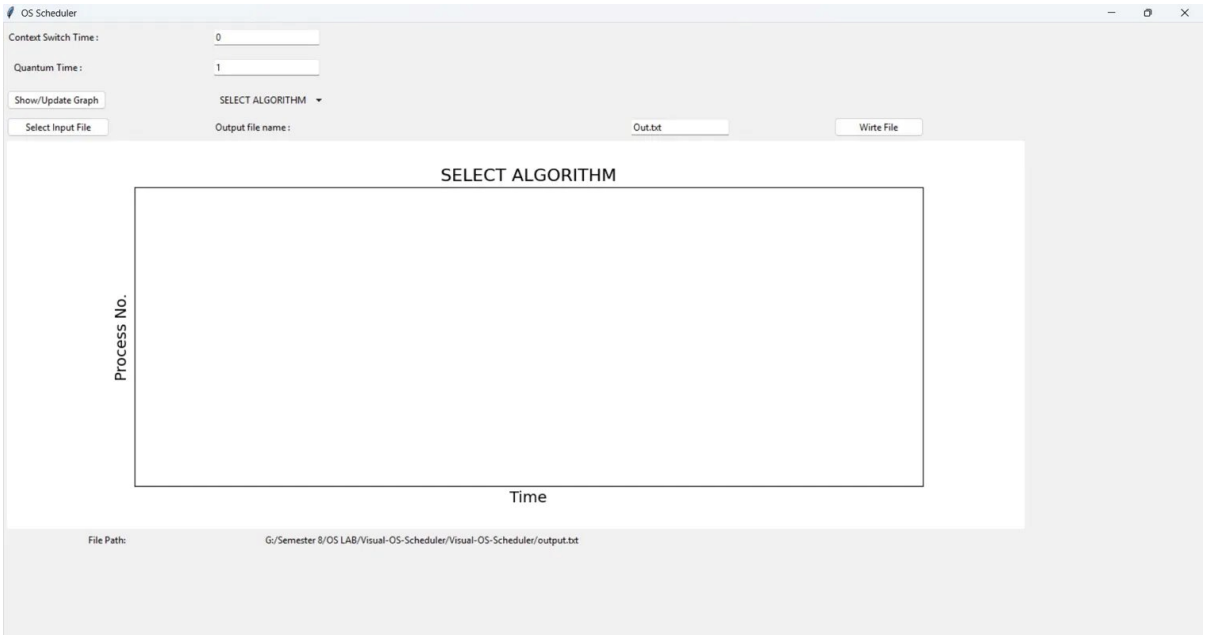

Figure 5: After clicking 'Select Input File', the loaded file path is shown at the bottom of the window.

Step 3: Select Algorithm

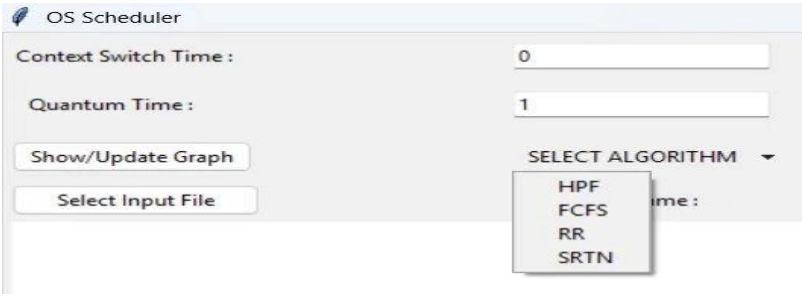

Figure 6: Algorithm selection dropdown showing HPF, FCFS, RR, and SRTN options.

Step 4: Show/Update Graph

After selecting RR with Quantum Time = 2 and Context Switch Time = 0, clicking 'Show/Update Graph' renders the Gantt chart showing Round Robin scheduling across all 10 processes.

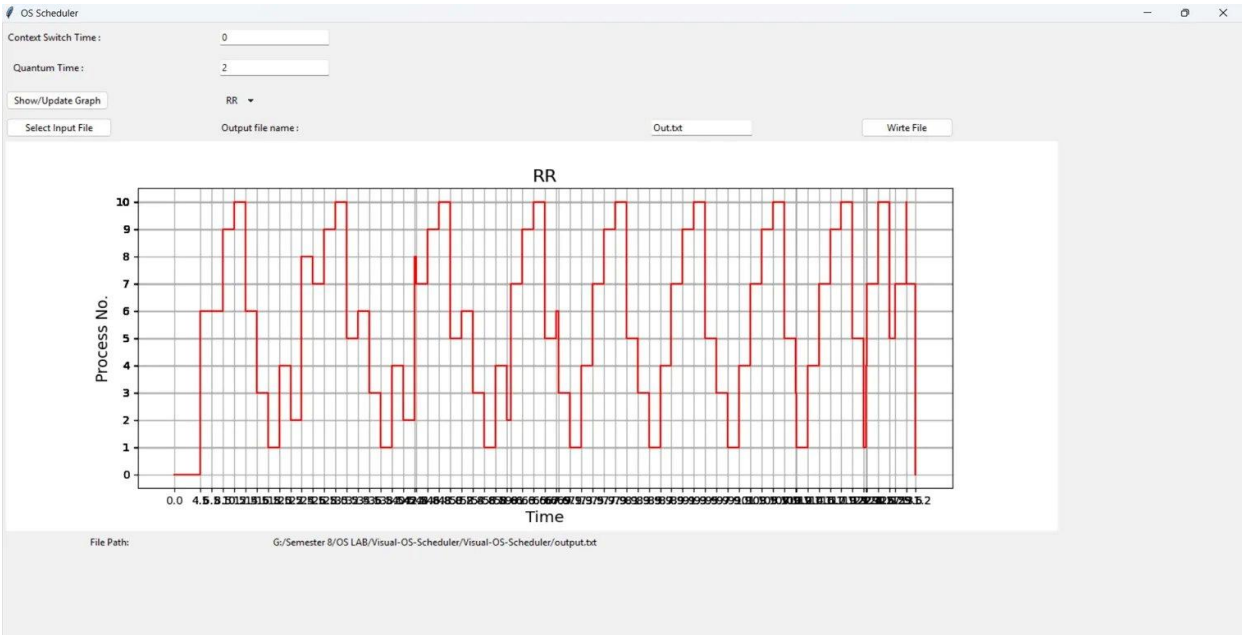

Figure 7: Round Robin Gantt chart with Quantum=2 and Context Switch=0, showing all 10 processes cycling through the CPU.

Step 5: Write File

Clicking 'Write File' saves the computed metrics (waiting time, turnaround time, weighted turnaround time) to Out.txt and displays a confirmation dialog.

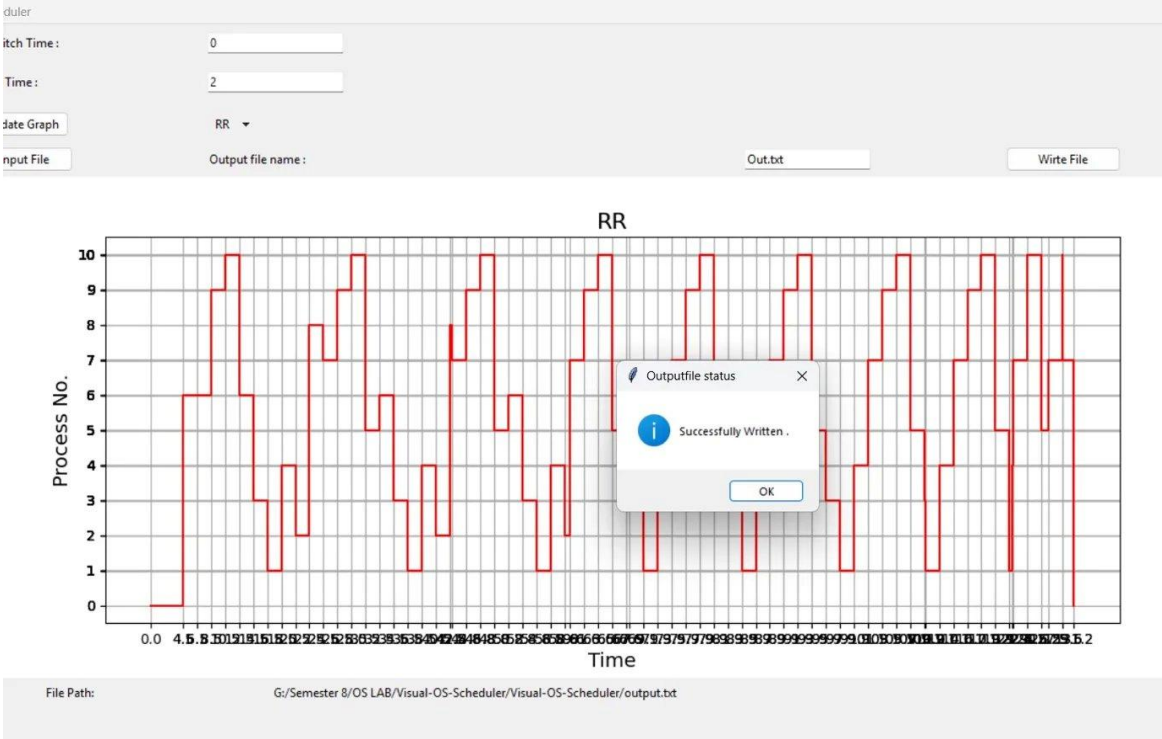

Figure 8: 'Successfully Written' confirmation dialog after exporting metrics to Out.txt.

Step 6: View Out.txt

The exported Out.txt file contains per-process metrics in tabular form along with aggregate averages for Turnaround Time and Weighted Turnaround Time.

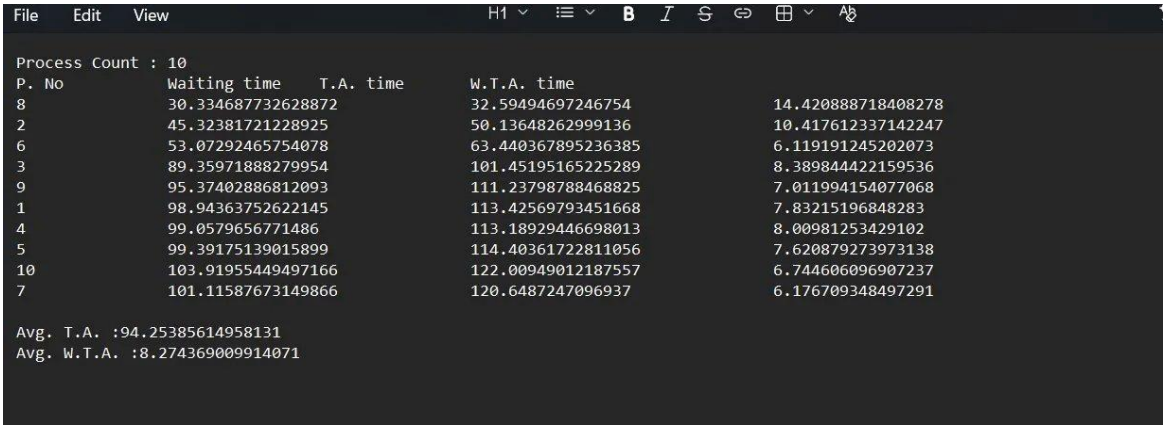

Figure 9: Contents of Out.txt showing process-level Waiting Time, T.A. Time, W.T.A. Time, and averages for the RR run.